

CG-HIK: Query-Adaptive Tail-Latency Routing for Kinematically Verified Online Inverse Kinematics

Anonymous Author(s)

Affiliation withheld for blinded review

Abstract

Online inverse kinematics (IK) is a repeated command-generation problem in which different queries can require sharply different numerical effort. A fixed solver sequence preserves robust coverage, but it can also make a simple continuation pay for computation needed only by a difficult, near-singular, or poorly initialized request. We present CG-HIK, a hybrid IK system that learns to select an entry into a shared numerical cascade. During development, every query is executed from every entry, producing action-complete observations of verified success, function evaluations (FEV), and repeated wall-clock latency. A compact multi-head model predicts shared terminal success and entry-specific P50 and P95 latency; its probability heads are calibrated. Among entries meeting frozen success and deadline criteria, the policy selects the lowest predicted P95. High-confidence portfolio failure can be rejected without numerical solving, whereas uncertainty is deferred to the complete cascade. The learned layer never issues a joint command. *Learning allocates solver effort per query; numerical geometry generates joint commands; deterministic verification governs acceptance.*

On fresh point-query tests, CG-HIK preserved feasible verified success while reducing mean FEV by 16.14% for Panda and 36.27% for UR5e relative to the Fixed robust cascade. Feasible P95 ratios were 0.7538 and 0.7427, and high-confidence rejection avoided 95.69% and 93.92% of fixed-cascade FEV on known-infeasible points. Median latency increased because batch-one prediction has a fixed cost. On a final one-shot evaluation of transition-rich trajectories, CG-HIK completed 41/80 Panda trajectories versus 40/80 for the Fixed hard-entry cascade, and 34/80 versus 34/80 on UR5e. Aggregate latency fell by 43.37% and 54.95%, and mean FEV fell by 45.16% and 65.12%. P50 rose by approximately 11.66% and 11.09%, while P95 fell by 12.04% and 21.94%. All accepted commands passed an independent replay of the same kinematic contract. The evidence supports query-adaptive solver allocation on exact URDF/Pinocchio kinematics; it does not establish collision safety, hard real-time bounds, or physical-robot performance.

Keywords: inverse kinematics; adaptive computation; algorithm selection; tail latency; damped least squares; hybrid learning and optimization; deterministic verification

1 Introduction

Robot task planners, teleoperation interfaces, vision-language-action policies, and trajectory generators commonly express motion as a stream of desired end-effector poses, whereas the actuator interface accepts joint-space commands. Inverse kinematics (IK) is therefore more than an isolated pose-to-joint regression. It is a continuous online interface that receives a desired pose, inherits a previous joint state and a finite control interval, chooses a solution branch, and either emits an admissible command or declines to do so. Consecutive requests can be close in Cartesian space yet differ in numerical conditioning, joint-limit proximity, and recoverability from the current branch. Conversely, a large motion can remain easy if the initializer lies in a favorable basin. Path-level and history-conditioned formulations increasingly recognize this temporal structure [1–4]. The engineering objective is consequently broader than minimizing average forward-kinematics error: an online IK system must preserve a clearly stated command contract, manage variable numerical work, expose failures, and control latency tails over a heterogeneous stream. A missed or discontinuous command also changes the state from which the next request is solved, so an apparently local error can alter an entire closed-loop trajectory. The relevant unit is therefore both the individual request, where a deadline is paid, and the trajectory, where accepted configurations become future initial conditions.

Analytical, Jacobian-based, and constrained numerical IK provide the geometric foundation for that interface. Structural decompositions can enumerate solution branches rapidly when a manipulator admits the required geometry [5–7], while pseudoinverse and damped-least-squares (DLS) updates provide general local solvers for arbitrary chains. Damping reduces the amplification of joint velocity near rank loss [8–10]; restart, variable-step, trust-region, quadratic-programming, and portfolio methods improve recovery from poor initialization or make limits explicit [11–15]. These approaches offer high precision, interpretable geometry, and constraints that can be checked independently. They should not be displaced merely because learned models are available.

Their practical limitation is different: a robust numerical portfolio often contains several increasingly expensive pathways, and a fixed entry or schedule cannot exploit the fact that only a subset of requests needs the full portfolio. This is not an argument against robust fallback. Rather, it asks whether the same coverage can be preserved while selecting a later entry when an early prefix is unlikely to help, or stopping before an expensive search when the complete portfolio is confidently predicted to fail.

Learned and generative IK address a complementary challenge: the inverse of a forward-kinematics map is generally a set, so a pose can correspond to discrete branches or a redundancy manifold [16]. Conditional flows, diffusion models, graph representations, invertible networks, and set predictors can represent multiple solutions more faithfully than a single deterministic regressor [17–22]. Reference-conditioned and sequential models further use the previous configuration to favor a locally coherent branch [3, 4, 23]. This literature has substantially improved candidate diversity, history continuity, path generation, and representation across robot structures. Its strength is amortized proposal and coverage. A learned output, however, is not automatically a valid online command: it can retain pose error, exceed a URDF joint limit, or require an inadmissible one-frame joint displacement. Learned prediction and command acceptance must therefore remain separate responsibilities. Moreover, proposal quality and system cost are not identical objectives. A candidate that is slightly better geometrically may not reduce the numerical iterations required downstream, while an inexpensive continuation can be preferable to a globally diverse proposal when the previous branch is already appropriate.

Hybrid learning–optimization systems exploit precisely this division of labor. Learning proposes a promising basin or trajectory, numerical optimization restores geometric precision, and explicit constraints or validity checks govern the final output. IKFlow and CycleIK support downstream numerical refinement [17, 24]; CppFlow couples generative path proposals to optimization and checking [2]; and recent diffusion-seeding, execution-aware, graph, and neuro-fuzzy systems similarly combine data-driven initialization with model-based optimization [25–29]. This proposal–refinement pattern is a strong basis for reliable IK because neither component is forced to perform the other’s role. In this paper, numerical geometry still generates every candidate command and a deterministic verifier still decides whether that command may be returned. The learned component is evaluated by the verified pathway it enables—including its inference cost and any fallback it triggers—rather than by proposal error in isolation. That systems boundary makes it possible to change computation allocation without changing the definition of a valid command.

A complementary gap remains at the level of computation allocation. Existing work primarily improves the proposal distribution, one numerical update, or a complete solver, while the surrounding hybrid system often retains a fixed candidate count, iteration budget, or pathway order. A fixed cascade can terminate early after a successful stage, but always starting from the same entry can still impose a predictably redundant prefix. Always beginning with a learned candidate can waste inference on an easy continuation; always beginning locally can waste a failed attempt before a hard pathway that was necessary from the outset. Seed retrieval and reselection improve basin choice [30], and failure detectors can trigger conservative fallback [31], but these mechanisms do not by themselves learn query-specific tail cost over all entries of one shared, deterministically verified cascade. We therefore focus on a narrow systems question: under an unchanged solver portfolio and command contract, which sufficient computation pathway should a particular query purchase? The emphasis on tail latency is deliberate. A controller may tolerate small median overhead if it avoids repeated long fallback episodes, whereas an average-only objective can conceal precisely the frames that dominate cumulative computation and interrupt an online loop.

This question connects online IK to per-instance algorithm selection [32, 33]. The key distinction is between robust *coverage* and uniform payment of worst-case *cost*. In our portfolio, *easy*, *medium*, and *hard* name entry points, not ground-truth difficulty classes. An easy entry starts with one previous-state local update and can escalate; a medium entry starts from the strongest learned proposal; a hard entry starts directly with the broader robust stage. All non-rejected entries retain the same terminal fallback. Thus the supervision required for routing is not a manually assigned easy/hard label. It is an action-complete matrix of what actually happened when every available entry processed the same query: whether the common verifier accepted a result, how many forward evaluations were spent, and what repeated wall-clock latency each pathway incurred. This matrix also exposes the empirical oracle gap: the cost remaining when the globally best fixed entry is used instead of the best observed entry for each query. A router is useful only if its own inference cost and selection regret are smaller than enough of that opportunity.

We instantiate this idea in CG-HIK. Development queries are executed through all three entries with balanced repeated timings. A compact two-layer model consumes history- and proposal-conditioned diagnostics and predicts a shared probability of terminal verified success, action-specific P50 and P95 latency, and the probability that every pathway fails. The shared success head reflects an observed and enforced property of the nested cascade: all entries terminate in the same robust fallback and had identical terminal semantic labels in the action-complete corpus. At runtime, an entry is eligible only when calibrated success clears a frozen threshold and predicted P95 lies within a frozen deadline. The policy then chooses the minimum predicted P95, subject to a conservative tie margin. A high-confidence failure prediction can reject without numerical solving; an out-of-distribution or otherwise uncertain request is deferred to the complete fixed cascade. This is action-complete

pathway supervision, not causal inference over unobserved potential outcomes. The implementation observes all candidate actions during development and chooses one at deployment. The two abstention outcomes are intentionally asymmetric. Reject means that the system has sufficient agreement to spend no numerical budget and emit no command; defer means the learned evidence is not adequate for an aggressive decision, so the complete numerical cascade is retained. This preserves uncertainty as a reason for more computation rather than silently converting it into a failure declaration.

The paper makes three contributions. First, it formulates online IK as query-adaptive resource allocation over a shared numerical portfolio, with a strict separation of roles: *learning allocates solver effort per query; numerical geometry generates joint commands; deterministic verification governs acceptance*. Second, it develops an auditable tail-latency router trained from action-complete pathway observations, with shared semantic-success prediction, action-specific latency quantiles, and distinct reject and conservative-defer semantics. Third, it evaluates the frozen system twice: on fresh independent point queries and on a final one-shot set of transition-rich closed-loop trajectories. The point study preserved feasible verified success while reducing mean FEV and P95, and it exposed adverse evidence—fixed batch-one overhead raised P50, the simple point-OOD score was weak, and defer produced no observed recovery. In the transition-rich study, CG-HIK matched or improved whole-trajectory completion, reduced aggregate latency by 43.37% for Panda and 54.95% for UR5e, and reduced mean FEV by 45.16% and 65.12%, while P50 increased on both robots. These results support adaptive effort rather than universal latency improvement, and they keep the claimed scope to exact kinematics and the implemented command contract. They also provide a direct closed-loop stress test of the central design choice: unlike long-lived local shortcuts, CG-HIK reevaluates the sufficient numerical entry for each request, so a previously accepted branch does not exempt the next query from pathway assessment.

2 Related Work

2.1 Numerical, Analytical, and Constrained IK

Classical IK spans exact structural solutions and general iterative updates. Geometric and analytical methods exploit screw structure and subproblem decomposition, providing rapid branch enumeration for suitable manipulators [5–7]. Jacobian inverse and pseudoinverse methods generalize across chains, while DLS and singularity-robust inverses stabilize updates near rank loss [8–10]. Modern numerical extensions vary the step, run complementary solvers, batch initializations, or encode viability constraints [11–15]. RelaxedIK shows how pose, continuity, collision, and singularity objectives can be treated jointly as online motion synthesis [1], while recent analysis distinguishes local numerical conditioning from the topology of branch transitions [34]. These methods establish that robust IK is often a portfolio problem. Our contribution is not a new local update or constraint formulation; it is a learned allocation policy over a fixed portfolio.

2.2 Learned Multi-Solution and History-Conditioned IK

Learned IK has progressed from point regression toward explicit representations of a set-valued inverse. Conditional normalizing flows, graph-equivariant generators, invertible modular networks, multiple-instance models, diffusion, and permutation-invariant set prediction improve diversity or structural transfer [17–22]. History-conditioned models add a reference configuration or predict a joint increment, making branch preference explicit rather than averaging globally separated solutions [3, 4, 23]. The diffusion and MimicIK records are treated here as preprints, and the sequential model as an online forthcoming record. Collectively, these methods show the value of learned proposal distributions. CG-HIK uses such a proposal upstream, but evaluates it through downstream numerical pathways and never treats a neural prediction as an accepted command.

2.3 Learned Proposals with Numerical Refinement

Hybrid pipelines combine the global coverage of learning with the precision and constraint handling of numerical geometry. IKFlow and CycleIK refine learned solutions numerically [17, 24]; CppFlow performs path-level proposal, search, and refinement [2]; and diffusion priors can seed or guide trajectory optimization [25, 26]. Execution-time-aware redundancy selection, graph warm starts, and neuro-fuzzy approximators likewise make learned outputs useful through their effect on a downstream solver [27–29]. We retain this cooperative architecture but shift the learned target from the joint command to the amount and entry of numerical work. This enables an interpretable system comparison: the primary baselines and CG-HIK share proposals, solvers, fallback, budgets, and verification, differing only in how the entry is selected.

2.4 Adaptive Computation and Solver Routing

Per-instance algorithm selection maps instance features to algorithms with different cost and success profiles [32]. Benchmark libraries formalize the corresponding instance–algorithm performance matrix and the distinction between a selected system and its empirical oracle [33]. Calibration and ensembles provide tools for controlling predictive confidence [35, 36], while runtime failure detection motivates conservative fallback under uncertainty [31]. Within IK, seed selection and reselection already demonstrate that basin choice and failure recovery affect runtime [30]. Among the reviewed systems, we did not identify the same combination of query-specific P95 routing, action-complete observations of a shared IK portfolio, semantic-success eligibility, explicit reject/defer separation, and deterministic command verification. We make the narrower claim that this combination fills a tail-latency routing gap; we do not claim the first use of learning to select an IK component.

3 Problem Formulation

3.1 Online query and command contract

At control step t , an IK query is

$$\xi_t = \left(x_t^d, q_{t-1}, \Delta t \right), \quad (1)$$

where $x_t^d = (p_t^d, R_t^d) \in SE(3)$ is the desired end-effector pose, $q_{t-1} \in \mathcal{Q}$ is the previously accepted configuration, and Δt is the command interval. A numerical pathway proposes q_t . The shared deterministic contract is

$$\begin{aligned} \mathcal{V}(q_t; \xi_t) = & \mathbf{1} \left[\|p(q_t) - p_t^d\|_2 \leq \epsilon_p \right] \\ & \wedge \mathbf{1} \left[d_R(R(q_t), R_t^d) \leq \epsilon_R \right] \\ & \wedge \mathbf{1} \left[q_{\min} - \epsilon_q \leq q_t \leq q_{\max} + \epsilon_q \right] \\ & \wedge \mathbf{1} \left[|\Delta q_t| \leq \dot{q}_{\max} \Delta t + \epsilon_v \right] \\ & \wedge \mathbf{1} [q_t \text{ is finite}]. \end{aligned} \quad (2)$$

The orientation distance d_R is the norm of the rotation logarithm. This is a kinematic command contract, not a universal safety certificate: it contains no collision, torque, contact, or tracking model.

3.2 Entry actions and tail-cost objective

Let $\mathcal{A} = \{\text{easy}, \text{medium}, \text{hard}\}$ denote entry actions into a nested numerical cascade. For query ξ and entry a , let $Y(\xi, a) \in \{0, 1\}$ be terminal verified success and let $T(\xi, a)$ be the observed action-pathway wall-clock latency, including proposal preparation, numerical solving, verification, and pathway framework overhead. The learned routing decision itself is excluded from these development labels because each action is forced during collection. Repeated measurements define conditional latency quantiles $Q_\tau(T \mid \xi, a)$. Fresh system measurements instead enclose the complete runtime, including the router’s fixed batch-one overhead.

The learned policy predicts $\hat{p}_s(\xi)$ and entry-specific $\hat{Q}_{0.50}(\xi, a)$ and $\hat{Q}_{0.95}(\xi, a)$. Given a calibrated success threshold $\pi_{\min}$ and deadline D , the eligible set is

$$\mathcal{A}(\xi) = \left\{ a \in \mathcal{A} : \hat{p}_s(\xi) \geq \pi_{\min}, \hat{Q}_{0.95}(\xi, a) \leq D \right\}. \quad (3)$$

When this set is nonempty, the idealized routing rule is

$$a^*(\xi) = \arg \min_{a \in \mathcal{A}(\xi)} \hat{Q}_{0.95}(\xi, a), \quad (4)$$

with a conservative tie rule described below. Empty eligibility does not itself imply infeasibility: the policy must distinguish confident portfolio failure from uncertainty.

3.3 Research questions

The experiments address three questions.

RQ1—Computation heterogeneity and predictability. Do actual pathway costs differ across queries and families, and can the learned router approach the empirical oracle in entry choice and latency regret?

RQ2—Fresh point-query system performance. Does the frozen system preserve verified success while reducing FEV, P95 latency, fallback work, and computation on known-infeasible requests, with P50, P99, OOD, and defer outcomes reported rather than omitted?

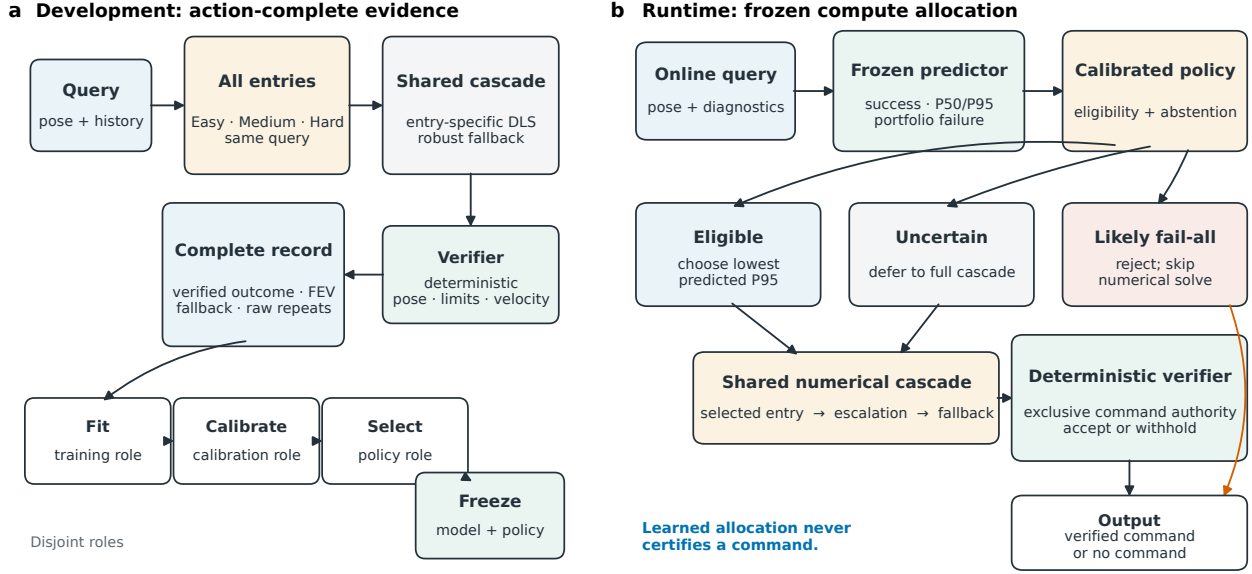


Figure 1: CG-HIK framework. During development (left), the same query is executed from every cascade entry and all verifier outcomes, FEV, fallback events, and raw timing repeats are retained. The compact predictor is fitted and calibrated on disjoint development roles. At runtime (right), the frozen model selects one eligible entry, rejects only high-confidence portfolio failure, or defers uncertainty to the complete cascade. Numerical solvers generate commands; the common deterministic verifier alone authorizes return.

RQ3—Fresh transition-rich trajectory performance. Relative to the Fixed hard-entry cascade, does CG-HIK preserve or improve whole-path completion while reducing aggregate latency, FEV, and upper frame-latency quantiles under method-specific closed-loop state?

4 CG-HIK Method

4.1 Shared Numerical Solver Portfolio

The three actions are entries into the same escalating cascade (Figure 1). The easy entry begins with one DLS update initialized at q_{t-1} . If verification fails, it escalates to medium and then hard. Medium begins with the best learned history-conditioned candidate and a bounded DLS refinement, then escalates to hard. Hard runs longer DLS attempts from the learned candidate and q_{t-1} and, if needed, a bounded trust-region-reflective fallback from heterogeneous previous, learned, and retrieved seed-bank configurations. Duplicated seeds are removed. The Fixed robust cascade always enters at easy; the Fixed hard-entry cascade starts directly at hard.

For a DLS step with task residual e and Jacobian J , the update has the weighted, damped form

$$\Delta q = J_w^\top \left(J_w J_w^\top + \lambda^2 I \right)^{-1} e_w, \quad (5)$$

with adaptive damping, joint-step clipping, and joint-limit clipping. The frozen portfolio uses the same tolerances, budgets, seed bank, fallback, and verifier for all methods (Table 1). Entry names describe execution order, not latent difficulty; an easy entry can still traverse the complete cascade after local failure.

4.2 History-Conditioned Candidate Proposal

A 5-member feed-forward ensemble proposes bounded joint increments conditioned on the previous configuration and desired pose. Each member contains 3 hidden layers of width 256 and predicts a normalized increment clipped to a maximum joint-space step. The ensemble is not used as an acceptance mechanism. Its best candidate is selected by a composite score over position and orientation residual, joint step, and joint-limit penalty, then supplied to medium/hard numerical refinement.

The routing feature vector contains 9 diagnostics available after candidate preparation: learned-candidate position and orientation residual, mean and maximum ensemble uncertainty, candidate Jacobian minimum singular value, candidate joint-limit margin, candidate-to-history joint step, and the position and orientation step from the current end-effector pose. These features describe both the proposal and the local query geometry. No feature relaxes the verifier.

4.3 Action-Complete Pathway Data Collection

For each development query, we execute easy, medium, and hard entries rather than inferring a label from a heuristic category. Every action retains repeated, Latin-rotated wall-clock measurements and the terminal verifier result, FEV, iterations, fallback status, executed stages, and command. An additional fixed-cascade audit arm verifies that entering at easy has identical semantics to the fixed primary baseline. The resulting matrix is action-complete: all alternative pathways were observed for the same query.

We occasionally use the term *counterfactual* for these alternative action records. It does not denote an estimate of unobserved potential outcomes. All actions were actually run during development, so *action-complete pathway supervision* is the more precise description. Data roles are separated by query identity: model fitting, probability/OOD calibration, and policy selection use disjoint development subsets, and no fresh test outcome participates in training or threshold selection.

4.4 Shared Success and Action-Specific Latency Prediction

The predictor is a SiLU multilayer perceptron with 9 inputs and 2 hidden layers of width 32; its hidden representation is shared by all heads. It has (i) one calibrated terminal-success logit broadcast across the three entries, (ii) six positive outputs for entry-specific P50 and non-crossing P95 latency, and (iii) a calibrated fail-all logit. P50 and P95 are optimized with pinball loss against all raw timing repeats, rather than against unstable single-run winners. Platt scaling is fitted only on the calibration role [35]. The success head is shared because action-complete development data showed exact terminal semantic invariance across entries under the common fallback; separate action-success heads would imply differences not present in the labels.

The latency heads serve a different purpose. Entry determines which prefix is skipped and therefore changes cost even when the terminal command is the same. Predicting both median and upper-tail pathway cost lets the selection objective prioritize deadline-relevant P95. The labels exclude the router’s own inference cost; the fresh end-to-end measurements include it and therefore expose its fixed P50 overhead.

4.5 Tail-Latency-Aware Routing Policy

After calibration, an entry is eligible when the shared success probability is at least the frozen minimum and its predicted P95 does not exceed the frozen 20 ms deadline. The policy selects the eligible entry with minimum predicted P95. If the predicted advantage over the most conservative eligible entry is smaller than the frozen tie margin, it keeps the conservative entry. This prevents small, noisy predicted differences from inducing an aggressive skip.

The policy, its threshold, the model weights, and all numerical budgets are frozen before either fresh evaluation. The Categorical routing baseline and Cartesian-step threshold baseline are retained only as development baselines: the former predicts entry labels, whereas the latter routes from target displacement alone. Neither is the final method.

4.6 Reject, Conservative Defer, and Deterministic Verification

The predictor also estimates the probability that the complete portfolio fails. A query can be rejected only when no numerical entry is eligible, the fail-all probability exceeds the frozen threshold, and the embedding is not flagged as OOD. Reject executes zero numerical stages and returns no command. When the embedding is OOD, or when no entry is eligible without high fail-all agreement, the policy defers to the complete Fixed robust cascade. Thus an uncertain query receives more computation rather than being declared infeasible.

For every numerical path, convergence is necessary but insufficient. The returned configuration must pass Eq. (2); otherwise the runtime escalates or returns failure. The learned system has no mechanism to override this decision. The fresh trajectory evaluator additionally reruns the same contract independently on every purportedly accepted command and counts any discrepancy as a contract violation.

4.7 Exact Batch-One Deployment

The trained predictor, normalization, calibrators, OOD transform, and policy constants are exported as a batch-one TorchScript module [37]. Release qualification compares eager and exported outputs, routing, command, FEV, fallback, and verification reason under frozen tolerances. At evaluation time the seed proposal runs on the GPU and the compact routing gate on the CPU, matching the sealed runtime specification. This exact export matters because algorithmic savings can be erased by per-query framework dispatch. It does not make the measurements hard real-time bounds; it makes the evaluated batch-one implementation explicit and reproducible.

Table 1: Shared solver portfolio and frozen configuration. All evaluated methods use these numerical and verification settings; only the entry-selection rule changes. Values are generated from the sealed configurations.

Component	Frozen setting	Role
Easy entry	Previous state	1 DLS iteration
Medium entry	Learned candidate	1 DLS iteration
Hard entry	1 learned candidate + previous state	25 DLS iterations per seed
Robust fallback	Previous + learned + up to 2 retrieved	TRF, max 100 FEV per seed
Verifier	Shared for every method	0.001 m / 0.008727 rad
Routing predictor	9 features; 32x32 MLP	shared success + action-specific P50/P95

5 Experimental Protocol

5.1 Systems, software, and frozen evidence roles

We evaluate Franka Emika Panda and Universal Robots UR5e serial manipulators using their exact URDF kinematics in Pinocchio [38]. SciPy supplies the trust-region-reflective fallback [39], and PyTorch supplies the learned modules and exact export [37]; scikit-learn utilities are used in the development calibration stack [40]. The benchmark does not step a physics simulator. All calls are batch one, CUDA synchronization is included at method boundaries where applicable, and each comparator has an independent runtime instance while sharing configuration semantics.

Development roles establish heterogeneity, train the model, calibrate probabilities and OOD score, and select the routing policy. Fresh point-query and fresh trajectory roles are immutable evaluation roles. Model weights, policy thresholds, solver budgets, fallback, and verifier are loaded from a sealed release and are not refitted or selected using evaluation outcomes.

5.2 RQ1: action-complete development study

The action-complete corpus contains 40000 queries across the two robots. For every query, each of the three decision entries is executed with 5 interleaved timing repeats. Risk-training, calibration, and policy selection identities are disjoint. We define the empirical oracle as the successful entry with the lowest observed P95; if no entry succeeds, the oracle is reject. Reported diagnostics are oracle-entry distribution, query-family by oracle-entry rates, the latency/FEV gap from fixed entries to the oracle, policy-oracle agreement, routing regret, and raw-sample quantile coverage. This stage diagnoses mechanism and predictability; it is not confirmatory evidence.

5.3 RQ2: fresh point-query evaluation

For each robot, the primary frozen point analysis contains 14000 feasible queries—10000 in-distribution and 4000 shifted feasible points—plus 2000 known-infeasible or one-frame-inadmissible points. Each feasible point has 3 raw solve calls per method. Training seed 17 is the primary released system; seeds 29 and 43 are sensitivity runs on the same test queries, not independent replications.

The primary comparator is the Fixed robust cascade, which shares candidate proposal, numerical stages, fallback, and verifier and always enters at easy. Metrics include verified success, FEV, fallback, P50/P95/P99, deadline misses, known-infeasible rejection and avoided FEV. We also report Mahalanobis OOD AUROC/AUPRC and defer recovery. The legacy frozen formal suite controlled the prespecified point and trajectory-completion hypotheses jointly by Holm. RQ2 reports the point outcomes; that legacy trajectory arm is distinct from the final transition-rich evaluation in RQ3. Separate finite operational gates are reported without being folded into the Holm family.

5.4 RQ3: one-shot transition-rich trajectories

The final evaluation contains 80 new closed-loop trajectories per robot, 20 from each of four preregistered families: smooth-fast-orientation-smooth, regular-near-singular-regular, central-joint-limit skim-return, and slow-high-curvature/high-speed-slow. Every trajectory has 150 frames at $\Delta t = 20$ ms. Identities were frozen before solver calls and checked by query hash, trajectory UID, numeric ID, and seed against registered development and prior evaluation lineages. Each method evolves its own previous joint state; therefore paired frame comparisons share the exogenous target and time index but not necessarily the complete solver input after branches diverge.

Table 2: Fresh feasible point-query results (primary frozen seed). Latency is in milliseconds. Success is determined by the shared kinematic command contract. Rows are generated from the sealed aggregate.

Robot	Method	Success (%)	Mean FEV	P50	P95	P99	Fallback (%)
Panda	Fixed robust cascade	99.03%	8.43	1.675	3.436	98.447	1.91%
Panda	CG-HIK	99.03%	7.07	1.950	2.590	97.768	1.91%
UR5e	Fixed robust cascade	100.00%	5.67	1.544	3.158	5.982	1.14%
UR5e	CG-HIK	100.00%	3.61	1.799	2.346	5.269	1.14%

Only three methods are executed: the Fixed robust cascade, the Fixed hard-entry cascade, and sealed CG-HIK. Every method is called once per frame in a balanced Latin order. The Fixed hard-entry cascade is the primary comparator because it was the strongest fixed entry in development; the Fixed robust cascade shows performance relative to the original full sequence. Whole-trajectory completion requires all 150 commands to be verified. The frozen gate requires CG-HIK completion not below hard, aggregate cumulative-latency and mean-FEV ratios no larger than 0.85 and 0.85, respectively; a P95 ratio no larger than 1.00; a P99 ratio no larger than 1.05; and zero contract violations. P50 is report-only.

5.5 Statistical and timing conventions

Point latency uses raw call-level quantiles because 3 interleaved solve calls are part of the frozen estimand. Development latency heads are trained on 5 raw repeats per entry. Trajectory latency is one retained solve call per method and frame; frames are temporally dependent, and the independent descriptive unit for completion is the whole trajectory. Cumulative latency is the exact sum over all fixed-length frames, and per-trajectory mean, median, and P95 are computed from 80 trajectory sums. Bootstrap tests in the point study resample the locked independent point unit and use the preregistered margins. We do not treat repeated calls or frames as independent replicates.

6 Results

6.1 RQ1: solver demand was heterogeneous and predictable

No fixed entry dominated the empirical oracle. Among successful development queries, Panda oracle assignments were 26.69% easy, 27.77% medium, and 45.54% hard; UR5e assignments were 23.41%, 25.36%, and 51.22%, respectively. Hard was the best single fixed entry, yet its mean excess over the oracle remained 0.137 ms for Panda and 0.154 ms for UR5e. Starting at easy left larger mean gaps of 0.391 and 0.478 ms, with the strongest hard preference in the hard-valid family.

Among successful, non-abstained queries in the development policy-selection role, CG-HIK selected the exact oracle entry for 47.42% on Panda and 53.88% on UR5e. Exact agreement is a strict criterion: an alternative entry can have practically indistinguishable cost. Mean routing regret was only 0.135 and 0.133 ms, with medians 0.016/0.000 ms and P95 0.467/0.605 ms. Raw-sample P95 coverage was close to its nominal level across entries. Figure 2 shows that the useful signal is a family- and query-dependent cost ordering, not a universal easy/medium/hard taxonomy.

6.2 RQ2: fresh point queries reduced P95 and numerical work

Table 2 and Figure 3 summarize the primary seed-17 point test. Feasible verified success was unchanged between CG-HIK and the Fixed robust cascade on both robots. Mean FEV fell by 16.14% for Panda and 36.27% for UR5e. P95 decreased from 3.436 to 2.590 ms for Panda and from 3.158 to 2.346 ms for UR5e, giving ratios 0.7538 and 0.7427. P99 improved materially for UR5e (0.8808) but was nearly unchanged for Panda (0.9931). P50 increased from 1.675 to 1.950 ms and from 1.544 to 1.799 ms, exposing the fixed cost of proposal features and batch-one routing. Observed fallback rates were unchanged at 1.91% for Panda and 1.14% for UR5e in both methods; the point study therefore does not support a fallback-rate reduction.

On the 2000 known-infeasible points per robot, command-reject recall was 95.65% for Panda and 93.95% for UR5e. Every command reject executed zero solver stages and zero FEV. Relative to running the fixed cascade, rejection avoided 95.69% and 93.92% of infeasible-point FEV. This result is evidence for avoiding futile computation on the constructed rejectable subset, not a proof of global unreachability.

The adverse abstention results bound that claim. Point-level OOD discrimination was weak: AUROC was 0.430/0.504 and AUPRC 0.210/0.230 for Panda/UR5e. Deferred requests were semantically identical to the

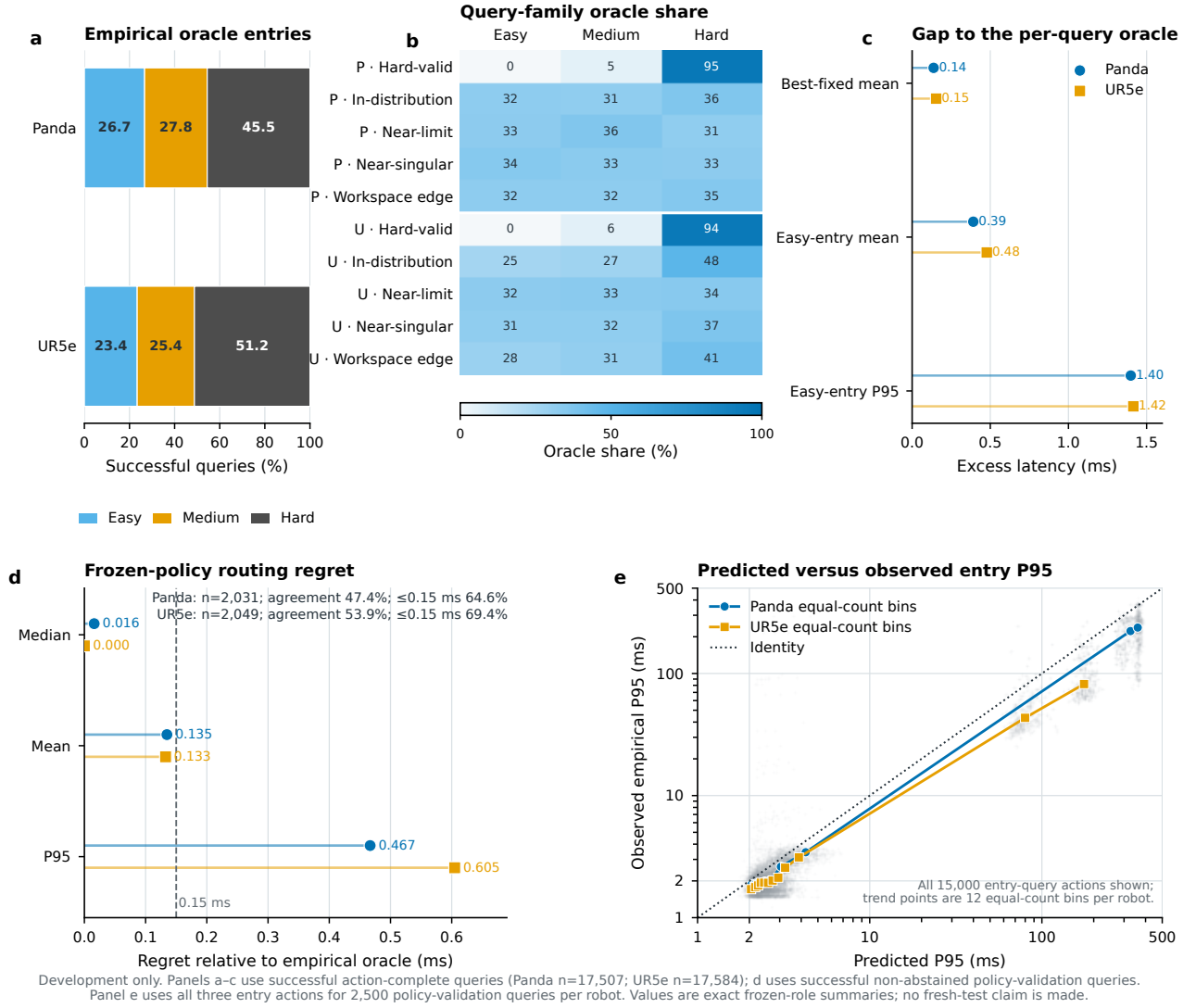


Figure 2: Computation heterogeneity and development predictability. (a) Empirical oracle-entry distribution among successful action-complete queries. (b) Oracle entry by query family. (c) Latency gaps from fixed entries to the empirical oracle. (d) Routing regret of the frozen policy. (e) Predicted versus observed P95 across entry actions. These are development diagnostics from a policy-selection role disjoint from fitting and calibration, not fresh-test claims.

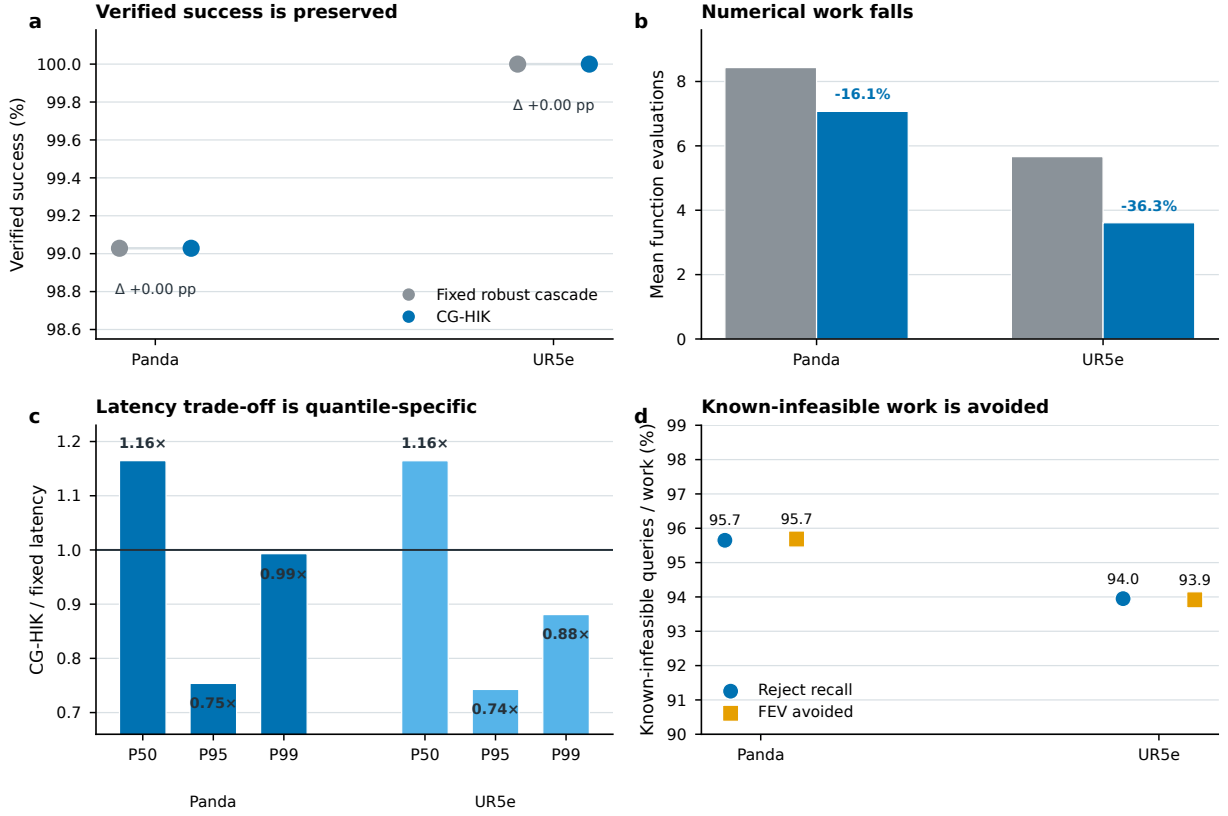
complete fixed cascade, but no successful defer recovery was observed (0.00%/0.00%). The prespecified point and legacy trajectory-completion margins passed joint Holm correction. A separate Panda operational condition failed because it required strictly positive improvement in OOD-feasible false rejection while both methods had zero false rejects; consequently the original point-study paper gate was false. We treat OOD as a conservative routing mechanism, not a demonstrated detector contribution.

6.3 RQ3: fresh transition-rich trajectories passed the final gate

The final one-shot trajectory evaluation passed all frozen gates for both robots (Table 3; Figure 4). Panda completion was 41/80 for CG-HIK and 40/80 for the Fixed hard-entry cascade; UR5e completion was 34/80 for both methods. No accepted-contract violation occurred in any of the three evaluated systems.

For Panda, aggregate cumulative latency fell from 472.04 to 267.31 s (43.37%), and mean FEV fell from 60.85 to 33.37 (45.16%). P50 increased from 1.651 to 1.844 ms (11.66%), while P95 decreased from 229.142 to 201.564 ms (12.04%). P99 was effectively tied: 353.570 versus 353.672 ms.

For UR5e, cumulative latency fell from 138.23 to 62.27 s (54.95%), and mean FEV fell from 35.23 to 12.29 (65.12%). P50 increased from 1.558 to 1.731 ms (11.09%), but P95 decreased from 38.104 to 29.745 ms (21.94%) and P99 from 50.821 to 39.085 ms (23.09%). Thus the result is a redistribution of computation away from expensive frames, not a uniform reduction of every latency quantile.



Fresh independent point evaluation: $n=14,000$ feasible and $n=2,000$ known-infeasible queries per robot. Points and bars are exact frozen-test aggregates; no replicate interval is implied. FEV denotes function evaluations.

Figure 3: Fresh point-query evidence. Panels compare feasible verified success, mean FEV, and P50/P95/P99 for the common fixed cascade and CG-HIK, followed by computation avoided on the known-infeasible subset. P50 overhead and the nearly unchanged Panda P99 are retained rather than hidden.

6.4 Benefits concentrated in expensive trajectory families

Family-level outcomes explain the aggregate savings (Figure 5). Among families with at least one completed trajectory on both robots, the regular-near-singular-regular family had the largest cumulative-latency reductions: 59.26% for Panda and 59.60% for UR5e. High-curvature/high-speed trajectories also benefited (44.09% and 20.47%), and smooth-orientation transitions showed more moderate reductions (29.62% and 9.44%). In contrast, the Panda joint-limit-skim reductions were -0.68% for cumulative latency and -0.58% for FEV; the negative values indicate a slight regression relative to hard. UR5e’s larger nominal joint-limit reduction occurred with zero completed trajectories under both methods and can reflect earlier failure, so it is not evidence of lower cost on successful paths. The Panda exception is important: when hard entry is already close to the effective per-query choice, prediction overhead and occasional routing choices can offset the saved work.

Panda’s high-curvature/high-speed family remained difficult for every method, with only 1/20 complete CG-HIK trajectories. The near-singular family, by contrast, combines high recovery cost with substantial entry heterogeneity, so skipping unproductive stages yields a larger return. The representative trace in Figure 5 shows how route changes coincide with bursts of latency and FEV; it is illustrative and not a separate statistical unit.

7 Discussion

7.1 Why query-adaptive computation works

The action-complete corpus shows that numerical demand is heterogeneous even when the portfolio, target contract, and terminal fallback are fixed. This gives the router a real decision to make. The hard entry is the best global fixed action, but easy and medium remain empirical winners for substantial subsets, while some families pay a large penalty for beginning too shallowly. CG-HIK does not need perfect oracle classification to help: many disagreements have small regret, and avoiding a few expensive prefixes or fallbacks can dominate the aggregate total.

This also clarifies the role of learning. The model is not approximating the final IK map in place of geometry. It

Table 3: Final fresh transition-rich trajectory results. Cumulative latency is the exact sum over all fixed-length frames. The Fixed hard-entry cascade is the primary comparator; the Fixed robust cascade contextualizes the original cascade.

Robot	Method	Completed/80	Cum. time (s)	Mean FEV	P50	P95	P99	Violations
Panda	Fixed robust cascade	41/80	487.64	63.79	1.653	236.316	355.565	0
Panda	Fixed hard-entry cascade	40/80	472.04	60.85	1.651	229.142	353.570	0
Panda	CG-HIK	41/80	267.31	33.37	1.844	201.564	353.672	0
UR5e	Fixed robust cascade	34/80	143.30	39.15	1.556	38.809	51.463	0
UR5e	Fixed hard-entry cascade	34/80	138.23	35.23	1.558	38.104	50.821	0
UR5e	CG-HIK	34/80	62.27	12.29	1.731	29.745	39.085	0

predicts how an already available solver portfolio is likely to behave. Because pathway outcomes were observed for the same development queries, the target is closer to operational cost than a hand-labeled difficulty class. The strongest evidence appears where failure recovery is expensive: near-singular and high-curvature/high-speed trajectories. Smooth or hard-aligned families leave less avoidable work.

7.2 Why P50 rises while cumulative latency and P95 fall

The router pays candidate-feature preparation and compact model inference on each query. For the large mass of frames that the hard entry already solves cheaply, this fixed cost raises the median by 11.66% for Panda and 11.09% for UR5e in the final trajectory test. Tail and cumulative metrics weight a different mechanism: difficult frames can trigger many numerical evaluations and expensive fallback. Selecting a sufficient later entry or rejecting a known-infeasible point avoids that work. Consequently, Panda and UR5e show large cumulative and FEV reductions, lower P95, and, for UR5e, lower P99, even though P50 rises. Panda P99 remains tied because a small set of hard numerical tails survives routing. These observations are why the objective and claims emphasize tail-aware resource allocation rather than universally faster queries.

7.3 Learning is not the acceptance authority

Every route ends at the same command contract. A high predicted success probability cannot authorize a command, and a learned proposal cannot bypass numerical refinement. The policy can decide to spend less work, more work, or no work; only the verifier can turn a numerical result into an accepted command. This separation makes failure observable and keeps the primary comparison interpretable. It also explains why zero contract violations are more informative than raw solver convergence: they show consistency with the implemented kinematic contract, not with an unspecified notion of safety.

7.4 Alternative temporal shortcuts

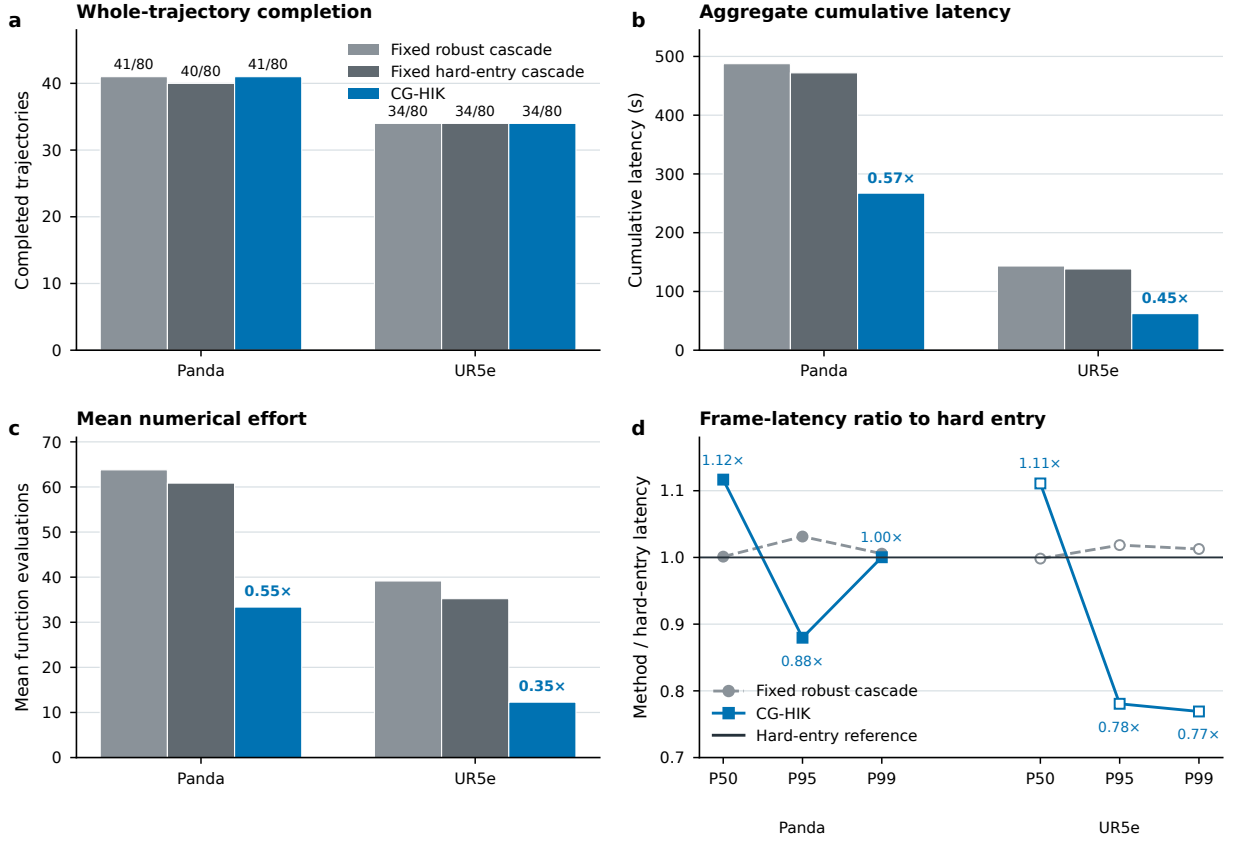
Development-only mechanism studies tested lighter per-frame gates and stateful local shortcuts. They confirmed that one-step local solving can produce long successful runs and very low median latency. They also revealed a structural risk: maintaining a locally verified branch changes subsequent q_{t-1} , and a later near-singular or transition-rich request may then become unrecoverable even when a hard-only history would succeed. Sparse hard re-anchoring reduced neither all long robust episodes nor all robot-specific completion losses under the preregistered gates. These studies are not formal results and introduce no component of the final method. They motivate retaining per-query pathway evaluation instead of committing to a temporal shortcut.

8 Limitations

First, the evidence uses exact URDF/Pinocchio forward kinematics and Jacobians. It is not a dynamics, physics-simulation, or physical-robot experiment. Robot assets were drawn from an Isaac Sim installation, but Isaac Lab dynamics were not executed. Collision, self-collision, contact, torque limits, actuator dynamics, sensor delay, and low-level controller tracking are outside the contract and must be evaluated before deployment.

Second, Eq. (2) is not a universal safety proof. It checks finite joint commands, pose residual, URDF joint bounds, and one-step velocity continuity only. Acceleration, jerk, swept-volume collision, force, and task-specific constraints are absent. Verified throughout the paper means verified against this exact kinematic contract.

Third, the Mahalanobis embedding score is a conservative switch, not a strong OOD detector. Point-level AUROC was 0.430 for Panda and 0.504 for UR5e, and no deferred point was recovered. OOD is therefore not a contribution supported by the present evidence.



One-shot transition-rich evaluation: $n=80$ trajectories and 12,000 frames per robot; each bar is an exact frozen-role aggregate. Ratios compare the same robot and frame quantile. P50 is report-only. FEV denotes function evaluations.

Figure 4: Final fresh transition-rich trajectory results. Completion, aggregate cumulative latency, mean FEV, and frame P50/P95/P99 are shown for the three frozen methods. Ratios annotate the primary CG-HIK-versus-Fixed hard-entry cascade comparison. P50 is report-only because routing incurs fixed batch-one overhead.

Fourth, latency is specific to the recorded software stack and hardware, with batch-one execution and explicit synchronization. Percentiles are empirical measurements, not hard real-time upper bounds. Fixed model inference raises P50, and rare numerical fallback still dominates extreme Panda tails. Hardware, compiler, and runtime changes require new measurement.

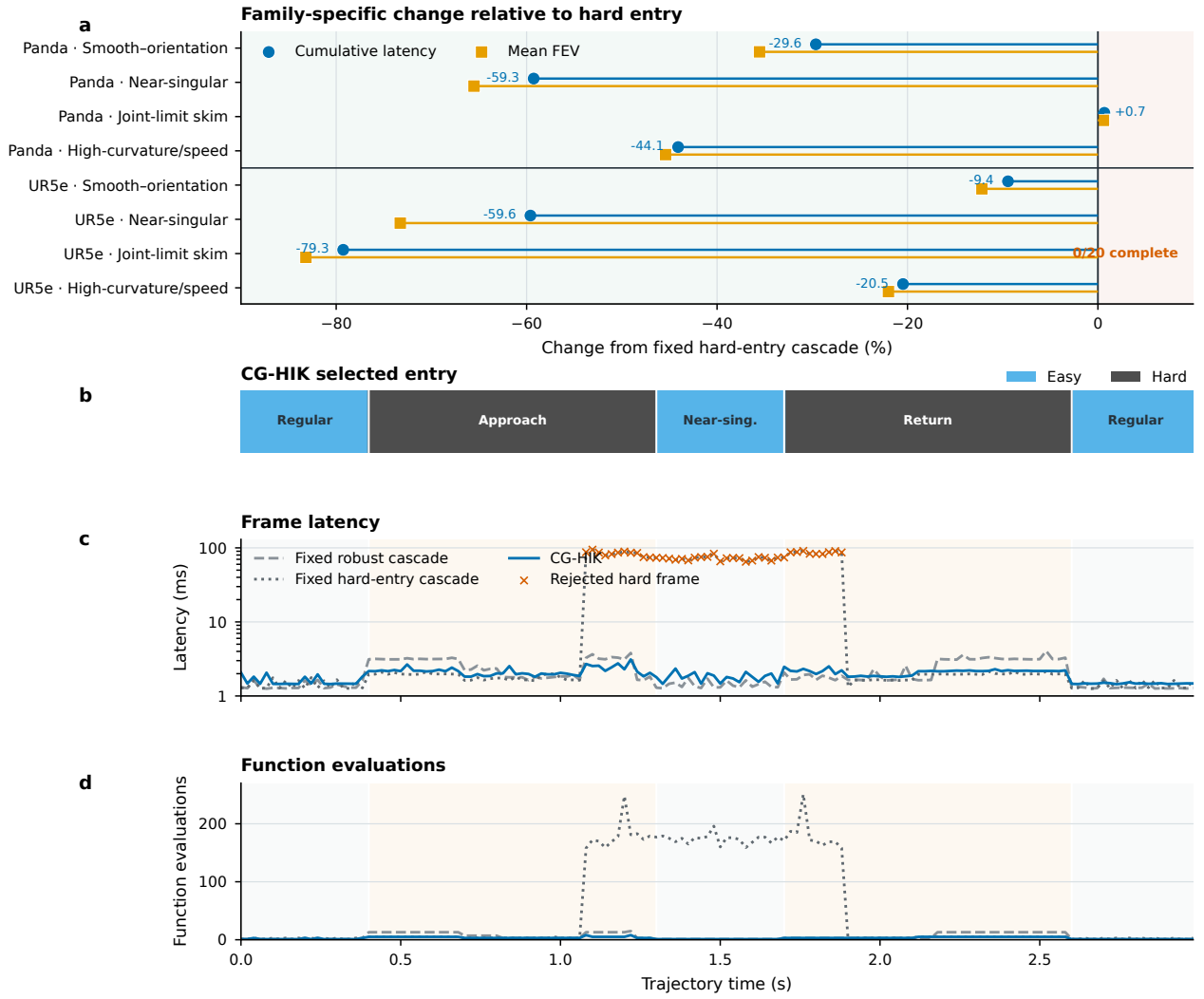
Fifth, the fresh transition set is deliberately stress-rich rather than a prevalence estimate for a deployment distribution. Panda high-curvature/high-speed completion remains very low, so lower cost must not be read as solving that regime. Closed-loop methods can follow different joint branches after an early command difference; their trajectory comparisons are policy-level evaluations under common exogenous targets, not identical-query microbenchmarks at every later frame.

Finally, models are robot-specific and evaluated on two manipulators. The study does not establish cross-robot transfer, collision-aware admissibility, or generalization to unseen hardware. Extending the contract and evaluating controller-level execution are separate next steps; they should preserve the current separation between learned allocation, numerical command generation, and deterministic acceptance.

9 Conclusions

CG-HIK treats online inverse kinematics as query-adaptive allocation of a shared numerical solver portfolio. Action-complete development observations make pathway cost and terminal success learnable without inventing difficulty labels. A compact tail-latency policy chooses among nested entries, distinguishes confident rejection from conservative defer, and leaves every joint command to numerical geometry and a common deterministic verifier.

The two frozen evaluations give a coherent but qualified result. On fresh point queries, CG-HIK preserved verified feasible success, reduced FEV and P95, and avoided most numerical work on known-infeasible requests; P50 increased, Panda P99 barely changed, point-OOD discrimination was weak, and defer showed no recovery. On the final transition-rich trajectories, completion matched or exceeded the strongest fixed entry, cumulative time fell by 43.37% and 54.95%, mean FEV fell by 45.16% and 65.12%, and the frozen final gate passed on both



a, $n=20$ fresh trajectories per robot and family; points are exact family aggregates. b–d, one deterministic Panda near-singular trajectory selected by the frozen rule (illustrative, not an independent statistical unit). Each method evolves its own closed-loop state; FEV denotes function evaluations.

Figure 5: Where routing saves computation. (a) Change in cumulative latency and FEV for each transition-rich family, CG-HIK relative to the Fixed hard-entry cascade. Negative values indicate savings; UR5e joint-limit trajectories had zero completions under either method. (b–d) A deterministic representative trajectory showing selected entry, frame latency, and FEV through the transition phases. The trace uses method-specific closed-loop state.

robots with zero contract violations. The improvement was largest where query demand and recovery cost were heterogeneous, not in every family or every latency quantile. The central conclusion is therefore precise: *learning allocates solver effort per query; numerical geometry generates joint commands; deterministic verification governs acceptance.*

Data and Code Availability

The repository contains the manuscript source, figure source data, deterministic evidence extractor, frozen configurations, release manifests, and hash-addressed evaluation outputs. The generated evidence snapshot records the source path and SHA-256 for every value used in the paper. A citable archival DOI will be added to the accepted-manuscript record.

Author Contributions

Author identities and CRediT roles are withheld for blinded review.

Funding

Funding information is withheld for blinded review.

Use of Generative Artificial Intelligence

OpenAI Codex assisted with language editing, LaTeX preparation, code review, and reproducible figure generation. Scientific values were regenerated from frozen artifacts and checked against independent raw-data audits. The authors reviewed the manuscript and remain responsible for all claims.

Conflicts of Interest

The authors declare no conflict of interest.

A Supplementary Evidence Boundaries

A.1 Point-test multiplicity and operational gate

The primary point system uses training seed 17. Two additional training seeds (29 and 43) reuse the same robot-specific test queries and therefore quantify sensitivity rather than independent replication. Prespecified point margins and a legacy trajectory-completion margin per robot form one joint Holm family. All margin nulls were rejected. The legacy trajectory arm is distinct from the final transition-rich evaluation. The separate operational gate included a strictly positive OOD-feasible false-reject improvement; Panda fixed and CG-HIK both had zero false rejects, so that condition and the point-study paper gate were false. No threshold or claim boundary was changed after observing this result.

A.2 Development-only baselines and mechanism studies

The Categorical routing baseline and Cartesian-step threshold baseline are development comparisons, not the final method. Additional lightweight and temporal shortcut studies are retained in the repository as development-only mechanism analyses. They are excluded from the fresh final trajectory comparison and from the three main result tables.

A.3 Reproducibility

The evidence builder reads frozen JSON/NPZ files, validates their recorded hashes, and emits the TeX macros and CSV files used by the tables and figures. The figure builder reads only those CSV files. Rebuilding the paper therefore does not execute an IK query, load a test solver, retrain a model, or alter an evaluation output.

References

- [1] Daniel Rakita, Bilge Mutlu, and Michael Gleicher. RelaxedIK: Real-time synthesis of accurate and feasible robot arm motion. In *Robotics: Science and Systems XIV*, 2018. doi: 10.15607/RSS.2018.XIV.043.
- [2] Jeremy Morgan, David Millard, and Gaurav S. Sukhatme. CppFlow: Generative inverse kinematics for efficient and robust cartesian path planning. In *2024 IEEE International Conference on Robotics and Automation*, 2024. doi: 10.1109/ICRA57147.2024.10611724.
- [3] Duc Tien Nguyen, Van Thanh Tri Nguyen, Duc Toan Luu, Truong Do, and Vu Linh Nguyen. Trajectory-based sequential learning for efficient and stable inverse kinematics of robotic manipulators. *Robotics and Autonomous Systems*, 205:105669, 2026. doi: 10.1016/j.robot.2026.105669. In press as of 2 September 2026; scheduled for the November 2026 issue.
- [4] Jiahao Yang, Shenhao Yan, Fan Feng, Chengsi Yao, Ge Wang, Zhixin Mai, Yiming Zhao, and Yatong Han. MimicIK: Real-time generative inverse kinematics from teleoperation with fk consistency, 2026. Preprint.
- [5] Alexander J. Elias and John T. Wen. IK-Geo: Unified robot inverse kinematics using subproblem decomposition. *Mechanism and Machine Theory*, 209:105971, 2025. doi: 10.1016/j.mechmachtheory.2025.105971.

- [6] Daniel Ostermeier, Jonathan K lzl, and Matthias Althoff. Automatic geometric decomposition for analytical inverse kinematics. *IEEE Robotics and Automation Letters*, 10(10):9964–9971, 2025. doi: 10.1109/LRA.2025.3597897.
- [7] Pablo C. Lopez-Custodio, Yuhe Gong, and Luis F. C. Figueredo. GeoFIK: A fast and reliable geometric solver for the IK of the Franka arm based on screw theory enabling multiple redundancy parameters, 2025. URL <https://arxiv.org/abs/2503.03992>. Preprint; first posted in 2025, latest revision in 2026.
- [8] Charles W. Wampler, II. Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods. *IEEE Transactions on Systems, Man, and Cybernetics*, 16(1):93–101, 1986. doi: 10.1109/TSMC.1986.289285.
- [9] Yoshihiko Nakamura and Hideo Hanafusa. Inverse kinematic solutions with singularity robustness for robot manipulator control. *Journal of Dynamic Systems, Measurement, and Control*, 108(3):163–171, 1986. doi: 10.1115/1.3143764.
- [10] Stefano Chiaverini, Bruno Siciliano, and Olav Egeland. Review of the damped least-squares inverse kinematics with experiments on an industrial robot manipulator. *IEEE Transactions on Control Systems Technology*, 2(2):123–134, 1994. doi: 10.1109/87.294335.
- [11] Patrick Beeson and Barrett Ames. TRAC-IK: An open-source library for improved solving of generic inverse kinematics. In *2015 IEEE-RAS 15th International Conference on Humanoid Robots*, pages 928–935, 2015. doi: 10.1109/HUMANOIDS.2015.7363472.
- [12] Jacinto Colan, Ana Davila, and Yasuhisa Hasegawa. Variable step sizes for iterative jacobian-based inverse kinematics of robotic manipulators. *IEEE Access*, 12:87909–87922, 2024. doi: 10.1109/ACCESS.2024.3418206.
- [13] Liangting Wu and Roberto Tron. IKSPARK: Obstacle-aware inverse kinematics via convex optimization, 2024. URL <https://arxiv.org/abs/2403.12235>. Preprint; first posted in 2024, latest revision in 2026.
- [14] Yachen Zhang and Ryo Kikuuwe. Ensuring viability: A QP-based inverse kinematics for handling joint range, velocity and acceleration limits, as well as whole-body collision avoidance. *Journal of Intelligent & Robotic Systems*, 112(1):16, 2026. doi: 10.1007/s10846-025-02335-z.
- [15] Cael Yasutake, Andrew H. Liu, Zachary Kingston, and Brian Plancher. HJCD-IK: Gpu-accelerated inverse kinematics through batched hybrid jacobian coordinate descent, 2025. URL <https://arxiv.org/abs/2510.07514>. Preprint; first posted in 2025, revised in 2026; accepted to the 2026 IEEE/RSJ International Conference on Intelligent Robots and Systems.
- [16] Bruno Siciliano. Kinematic control of redundant robot manipulators: A tutorial. *Journal of Intelligent and Robotic Systems*, 3(3):201–212, 1990. doi: 10.1007/BF00126069.
- [17] Barrett Ames, Jeremy Morgan, and George Konidakis. IKFlow: Generating diverse inverse kinematics solutions. *IEEE Robotics and Automation Letters*, 7(3):7177–7184, 2022. doi: 10.1109/LRA.2022.3181374.
- [18] Oliver Limoyo, Filip Mari , Matthew Giamou, Petra Alexson, Ivan Petrovi , and Jonathan Kelly. Generative graphical inverse kinematics. *IEEE Transactions on Robotics*, 41:1002–1018, 2025. doi: 10.1109/TRO.2024.3521862.
- [19] Mochammad Rizky Diprasetya, Johannes P ppelbaum, and Andreas Schwung. KineNN: Kinematic neural network for inverse model policy based on homogeneous transformation matrix and dual quaternion. *Robotics and Computer-Integrated Manufacturing*, 94:102945, 2025. doi: 10.1016/j.rcim.2024.102945.
- [20] Youn-Jae Go and Jun Moon. EMIKNet: Expanding multiple-instance inverse kinematics network for multiple end-effector and multiple solutions. *IEEE Access*, 13:25087–25096, 2025. doi: 10.1109/ACCESS.2025.3539022.
- [21] Zeyu Zhang and Ziyuan Jiao. IKDiffuser: A diffusion-based generative inverse kinematics solver for kinematic trees, 2025. URL <https://arxiv.org/abs/2506.13087>. Preprint; first posted in 2025, revised in 2026; under review.
- [22] Duc Tien Nguyen, Van Thanh Tri Nguyen, Truong Do, and Vu Linh Nguyen. Multi-solution inverse kinematics for robotic manipulators via permutation-invariant set prediction. *IEEE Robotics and Automation Letters*, 11(8):9803–9810, 2026. doi: 10.1109/LRA.2026.3703280.
- [23] Jacket Demby’s, Ramy Farag, and Guilherme N. DeSouza. Inverse kinematics of robotic manipulators using a new learning-by-example method. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 9534–9541, 2024. doi: 10.1109/IROS58592.2024.10802048.

- [24] Jan-Gerrit Habekost, Erik Strahl, Philipp Allgeuer, Matthias Kerzel, and Stefan Wermter. CycleIK: Neuro-inspired inverse kinematics. In *Artificial Neural Networks and Machine Learning – ICANN 2023*, volume 14254 of *Lecture Notes in Computer Science*, pages 457–470. Springer, 2023. doi: 10.1007/978-3-031-44207-0_38.
- [25] Huang Huang, Balakumar Sundaralingam, Arsalan Mousavian, Adithyavairavan Murali, Ken Goldberg, and Dieter Fox. DiffusionSeeder: Seeding motion optimization with diffusion for rapid motion planning. In Pulkit Agrawal, Oliver Kroemer, and Wolfram Burgard, editors, *Proceedings of the 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 4392–4409. PMLR, 2025. URL <https://proceedings.mlr.press/v270/huang25f.html>.
- [26] João Carvalho, An Thai Le, Piotr Kicki, Dorothea Koert, and Jan Peters. Motion planning diffusion: Learning and adapting robot motion planning with diffusion models. *IEEE Transactions on Robotics*, 41: 4881–4901, 2025. doi: 10.1109/TRO.2025.3593109.
- [27] Yucheng Tang, Xi Huang, Yongzhou Zhang, Tao Chen, Ilshat Mamaev, and Björn Hein. ETA-IK: Execution-time-aware inverse kinematics for dual-arm systems. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 7504–7511, 2025. doi: 10.1109/IROS60139.2025.11247583.
- [28] Meenalochani Jayabalan, Karunamoorthy Loganathan, and Palanikumar Kayaroganam. A novel hybrid ik architecture for robotic arms: Iterative refinement of soft-computing approximations with validation on abb irb-1200 robotic arm. *Machines*, 14(3):292, 2026. doi: 10.3390/machines14030292.
- [29] Ali Jlidi, Rabab Benotsmane, and László Kovács. Explainable graph neural networks towards data-driven inverse kinematics in industrial robot motion planning. *Electronics*, 15(14):3071, 2026. doi: 10.3390/electronics15143071.
- [30] Xinyi Yuan, Weiwei Wan, and Kensuke Harada. IKSel: Selecting good seed joint values for fast numerical inverse kinematics iterations. *IEEE Transactions on Automation Science and Engineering*, 23:4410–4427, 2026. doi: 10.1109/TASE.2026.3659225.
- [31] Chen Xu, Tony Khuong Nguyen, Emma Dixon, Christopher Rodriguez, Patrick Miller, Robert Lee, Paarth Shah, Rares Andrei Ambrus, Haruki Nishimura, and Masha Itkina. Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies. In *Proceedings of Robotics: Science and Systems XXI*, Los Angeles, CA, USA, 2025. doi: 10.15607/RSS.2025.XXI.073.
- [32] John R. Rice. The algorithm selection problem. In Morris Rubinoff and Marshall C. Yovits, editors, *Advances in Computers*, volume 15, pages 65–118. Academic Press, 1976. doi: 10.1016/S0065-2458(08)60520-3.
- [33] Bernd Bischl, Pascal Kerschke, Lars Kotthoff, Marius Lindauer, Yuri Malitsky, Alexandre Fréchet, Holger Hoos, Frank Hutter, Kevin Leyton-Brown, Kevin Tierney, and Joaquin Vanschoren. ASlib: A benchmark library for algorithm selection. *Artificial Intelligence*, 237:41–58, 2016. doi: 10.1016/j.artint.2016.04.003.
- [34] Ivan Boschi, Alessandro De Toni, Roberto Di Leva, Edoardo Idà, and Marco Carricato. Handling transitions across singularities for UR-like serial robots. *IEEE Robotics and Automation Letters*, 11(3):2714–2721, 2026. doi: 10.1109/LRA.2026.3653295.
- [35] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pages 1321–1330. PMLR, 2017. URL <https://proceedings.mlr.press/v70/guo17a.html>.
- [36] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/9ef2ed4b7fd2c810847ffa5fa85bce38-Abstract.html.
- [37] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.
- [38] Justin Carpentier, Guilhem Saurel, Gabriele Buondonno, Joseph Mirabel, Florent Lamiriaux, Olivier Stasse, and Nicolas Mansard. The pinocchio c++ library: A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives. In *2019 IEEE/SICE International Symposium on System Integration*, pages 614–619, 2019. doi: 10.1109/SII.2019.8700380.

- [39] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- [40] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011. URL <https://www.jmlr.org/papers/v12/pedregosa11a.html>.